

MAP - Lecture 1
(Java basics)

5 oct 2026

Lab 35%

Seminar 5%

Final exam 60%
 { theoretical 25%
 { practical 35%

! GitHub commits

C code $\Rightarrow$ Rust ('28)

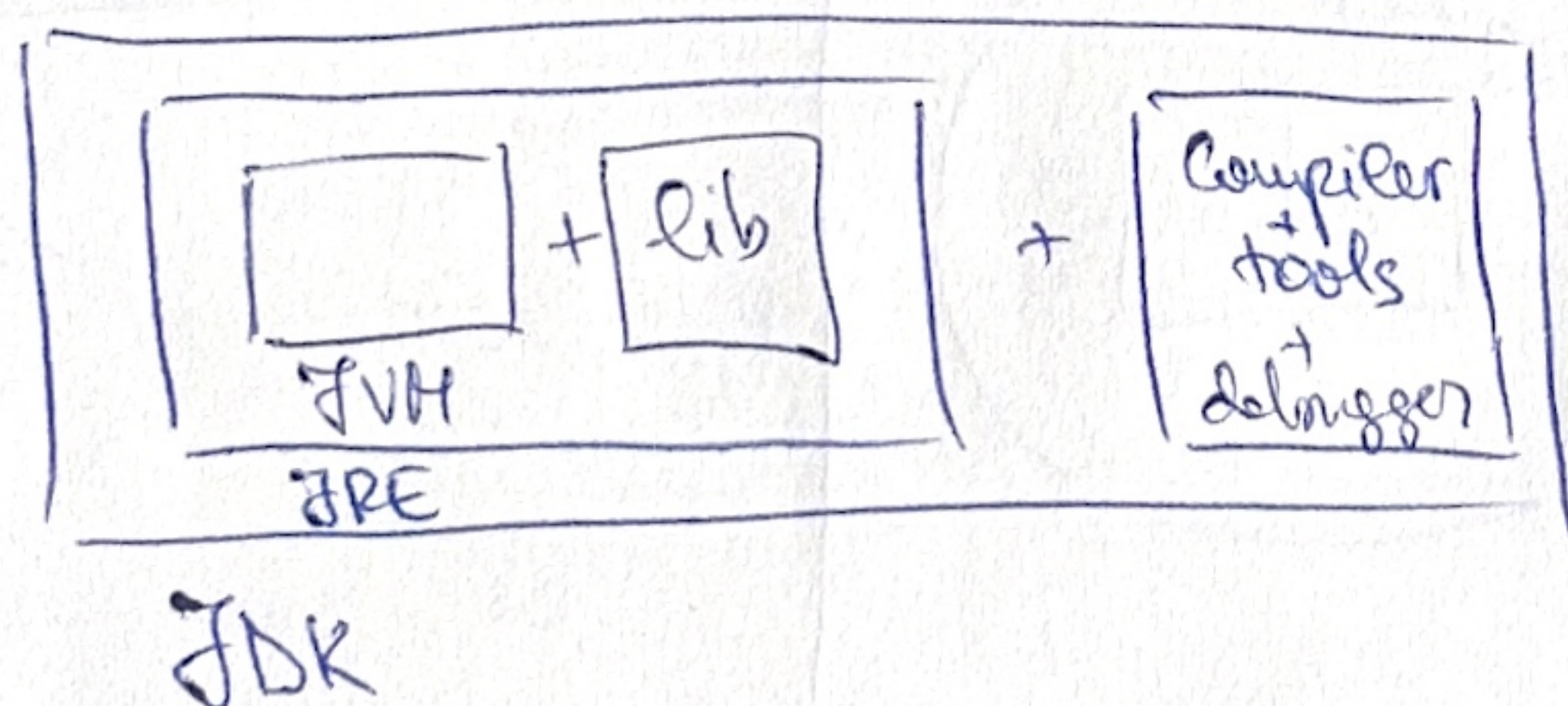
! Explain code

1) Java - compile once, run anywhere
- compiler tries to catch an error before execution

dev.java/learn } resources
"Thinking in Java"

bytecode $\sim$ assembly

JVM = interpreter $\leftarrow$ $\neq$ for each OS



compile + run : (Java 11+) : java Test.java

signature = name of function + param. list $\rightarrow$ return type is not part of the signature!

! Command-line args are always strings. $\rightarrow$ need conversion!

jshell (Java 9+)

ex A1:

```
Scanner scanner = new Scanner(System.in);  
String input = scanner.nextLine();  
int option = Integer.parseInt(input);
```

- 2) { Encapsulation $\rightarrow$ hide the implementation (HIDE DATA) (ex: I don't care if I use a stack or a LL. for push/pop)
getters, setters $\rightarrow$ control access because fields are private
Abstraction $\rightarrow$ HIDE DETAILS (interfaces, abstract classes)
Inheritance $\rightarrow$ a class acquires the fields/methods of another class (IS-A)
Polymorphism $\rightarrow$ same method call behaves $\neq$ depending on object TYPE.

• more classes in the same file, but only one class can be public.

ACCESS MODIFIERS:

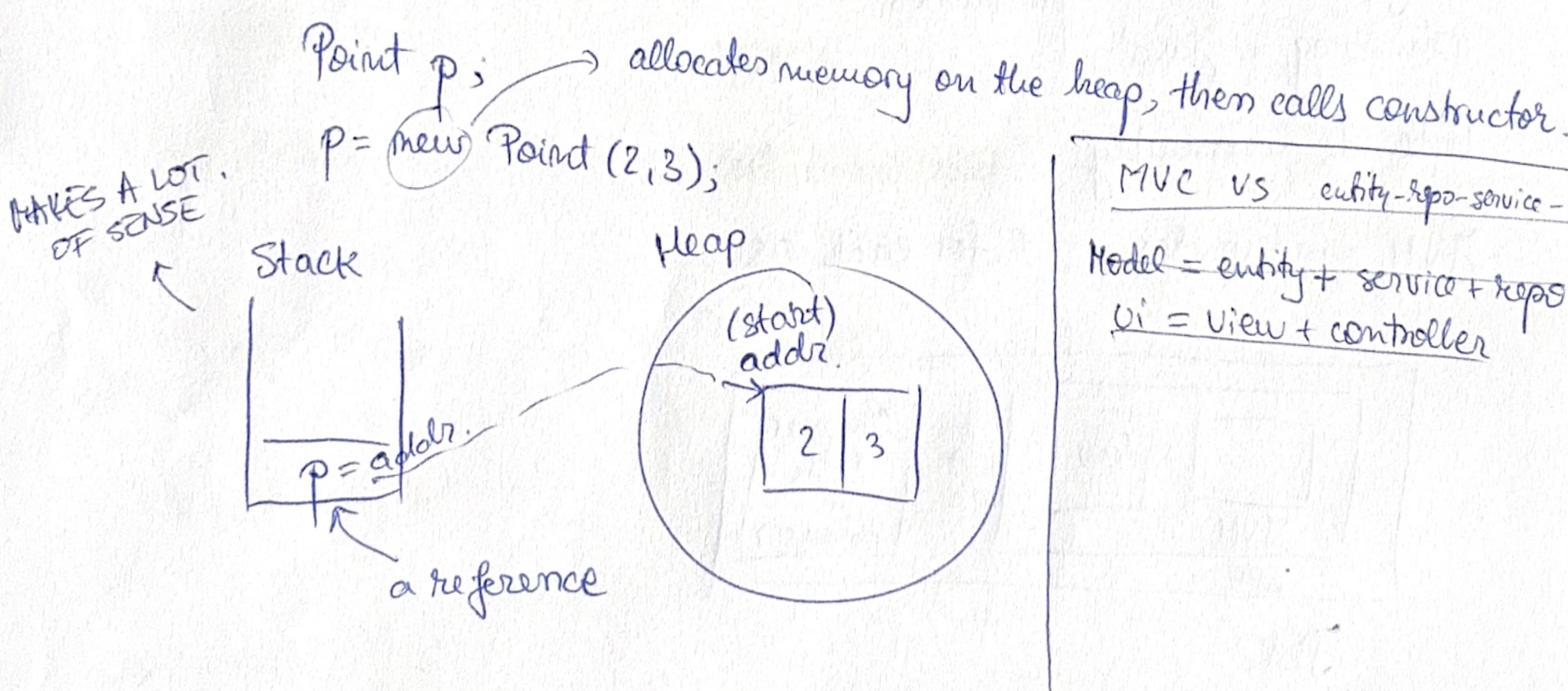
- private
- public
- protected
- none $\Rightarrow$ package-level

MVC Architecture - packages for each responsibility

- Model
- View
- Controller

If not initialized, the compiler will assign to those variables the default type. (ex: 0 for int)

CONSTRUCTOR - has the same name as the class
- does the initialization



MVC VS entity-repo-service-logic

Model = entity + service + repos

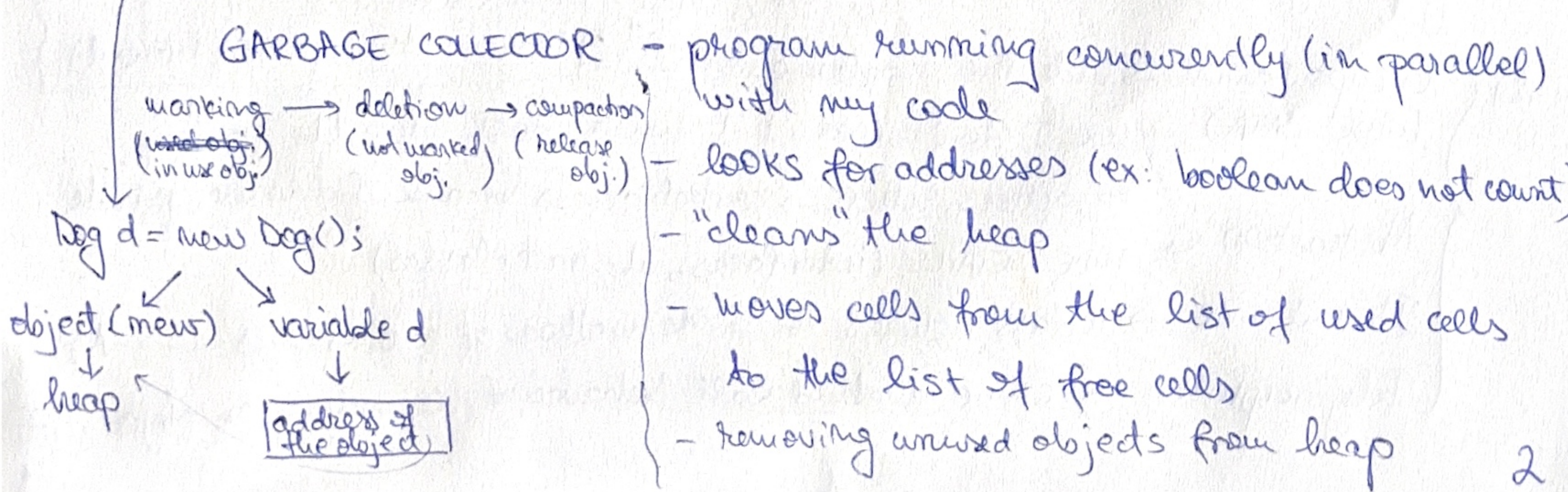
Vi = View + controller

OVERLOADING $\leftarrow$ methods with the same name, $\neq$ signature

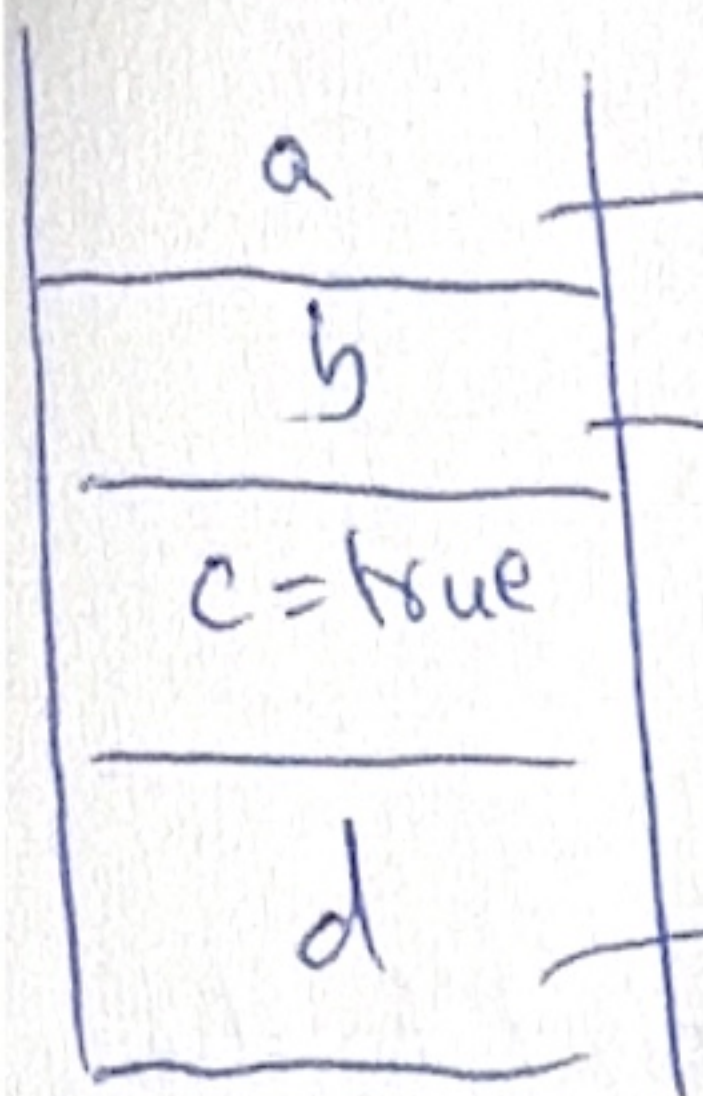
THIS $\leftarrow$ refers to the current obj. ("receiver")

Parameter Shadowing: this.real = real;

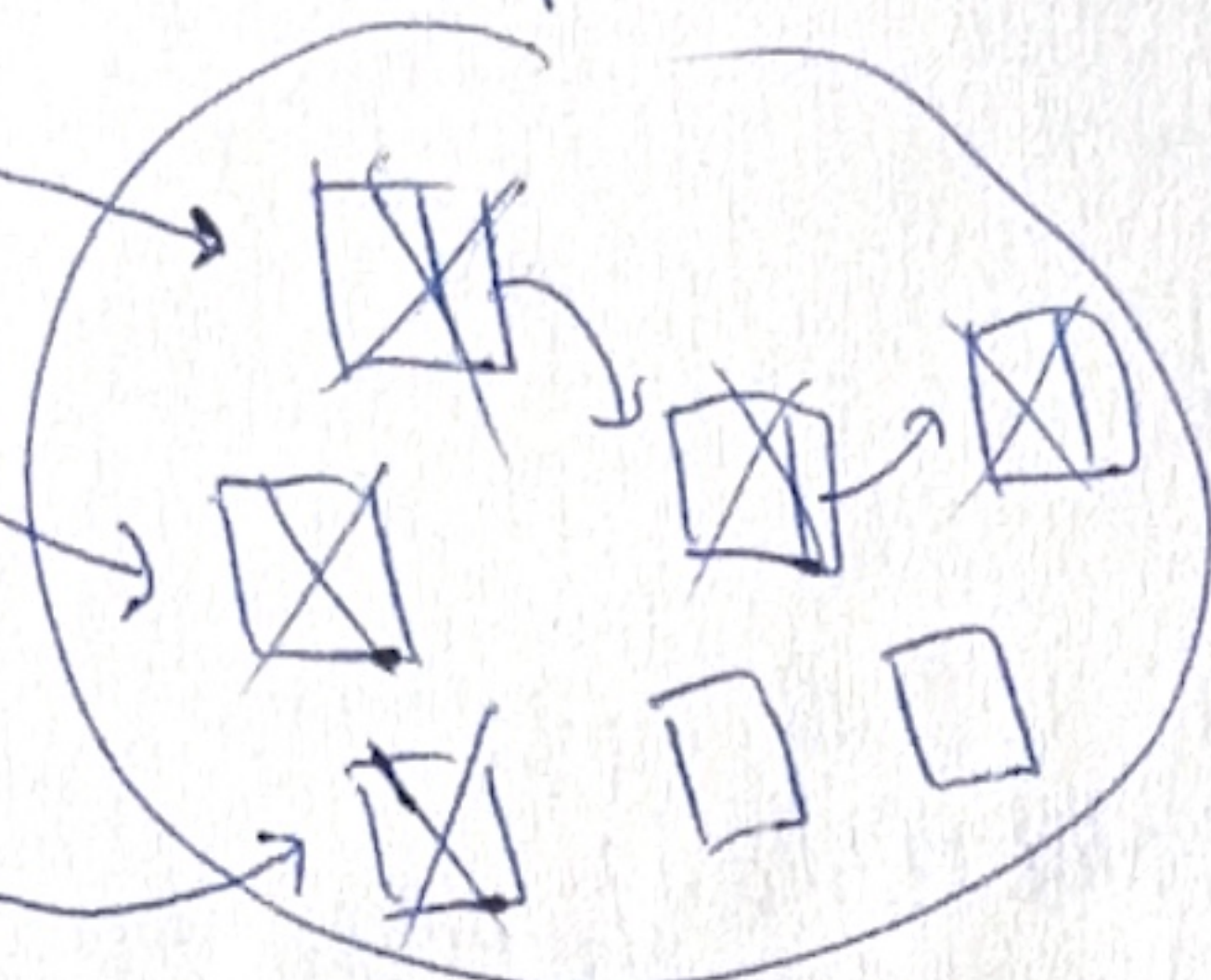
? A variable of a class type holds a reference, not the obj. itself.



Stack



Heap



OS

- 1) list of used cells
- 2) list of free cells

→ SHARED BY ALL OBJECTS

STATIC fields - belongs to a class, not an object

↳ See example Proodus.java

(every time the constructor gets called, ~~#~~ will be incremented)

ex: counter

the static field ~

OPERATOR OVERLOADING - not allowed in Java.

Calling static methods:

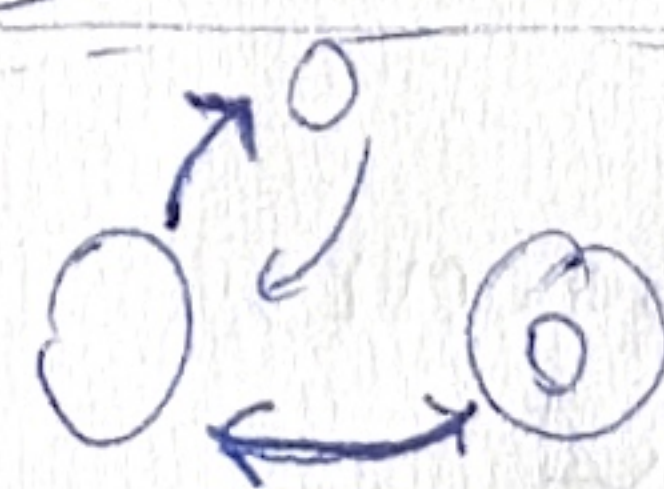
Complex c = Complex.sum(a, b);

primitive ^{data} types ← stored on the STACK

reference types

```
class Tree {
    static int cnt = 0;
    String name;
    Tree(String m) {
        name = m;
        cnt++;
    }
    new Tree("a");
    new Tree("b");
    System.out.println(
        Tree.count);
}
```

see?
NO OBJECT!
CLASS
IS USED!



var count = 10

↪ infer the type of a local variable / object.

→ cannot be re-initialized with another type

4) ARGUMENTS ∞ PASSED BY VALUE

→ See 2-4 example

5) ARRAYS

• unidim. array

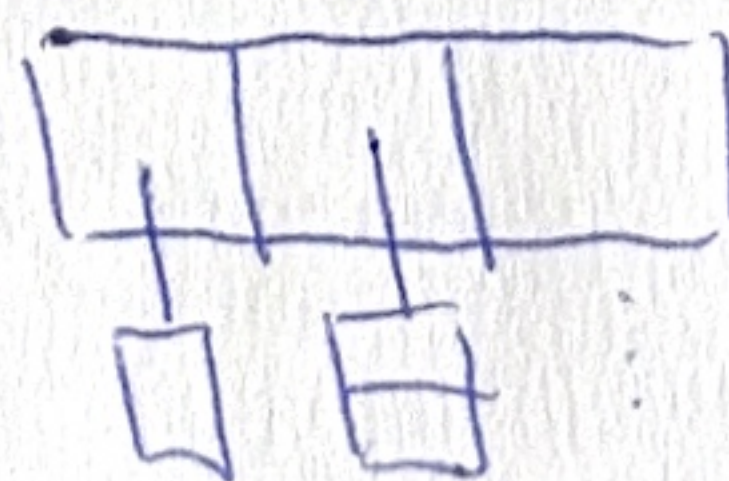
for-each: for (Type elem:) instruction

• multidim. array

rectangular



non-rectangular



ex: var count = 10; //int
count = 20; //OK
count = "hi"; //NOT OK
var x = null; //WRONG

NULL HAS NO TYPE OF ITS OWN!

- array of objects

java.util.Array ← algorithms.

{ t changes (immutable) $\leftarrow$ abc12312
 s remains the same $\leftarrow$ abc123 } even though they point to the same memory zone?

StringBuilder better than String

↓ ↓

mutable immutable

yield ← on that branch only (6)

STRINGS
↓
immutable
↓

```
ex :  
String s = "abc123";  
String t = s;  
t = "abc1234"
```

ARRAUS
↓
not immitable
↓

```
ex:  int[] a = {1,2,3};  
      int[] b = a;  
      b[0] = 99;
```

$s \Rightarrow abc123$

$t \nearrow$
 $\searrow$ abc1234

t gets changed
S remains the same

$$a \rightarrow [X, 2, 3]$$

$$b \nearrow$$

$$\Downarrow$$

~~a gets~~
b[0] gets changed
a gets changed as well

THIS LOOKS LIKE AN
EXAM QUESTION!